

06_data_a

Section 5: Data Analytics Interview

This section drills the analyst craft that sits under Helios: writing correct GA4 BigQuery SQL (UNNEST, sessionization, dedup, funnels, cohorts, mix-vs-rate aggregation) and designing metrics that survive scrutiny (ratio discipline, governed definitions, reconciliation, denominators, guardrails). The through-line is that in Helios the analyst writes none of this SQL at runtime — `semantic-mcp` emits it from a governed registry — so every answer doubles as a spec for what that registry and the dbt models must encode correctly once, forever.

SQL

Q1. The GA4 BigQuery export stores `event_params` as a repeated record. How do you pull a single parameter like `ga_session_id` out of it, and why is the syntax shaped that way?

Ideal answer. `event_params` is an `ARRAY<STRUCT<key STRING, value STRUCT<string_value, int_value, float_value, double_value>>>`, so a parameter is one row inside a nested array, not a column. I extract it with a correlated subquery over `UNNEST`, picking the right typed slot: `(SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key = 'ga_session_id')` AS `ga_session_id`. The subquery form is preferable to a top-level `CROSS JOIN UNNEST` when I want many params side-by-side on one event row, because each subquery returns a scalar and avoids fanning the event out into one row per param. The cardinal mistake is reading the wrong value slot (e.g. `string_value` for a numeric `ga_session_id`, which yields `NULL`), so I always match the type to the parameter. **Why Helios demonstrates it.** This exact pattern lives in `stg_ga4__event_params`, which unnests `event_params` once into a long key/value table so no downstream model re-implements `UNNEST`; the `get_event_param` macro centralizes the typed-slot extraction. **Follow-ups.** When would you prefer `CROSS JOIN UNNEST` over the scalar subquery? / How does `UNNEST(event_params) WITH OFFSET` help and when do you need it? / What happens to the event row if a param key is genuinely absent?

Q2. Define a GA4 session in SQL. What is the session key and why isn't `ga_session_id` alone enough?

Ideal answer. `ga_session_id` is only unique within a single user, so the session grain is the pair `(user_pseudo_id, ga_session_id)`; using `ga_session_id` alone collides sessions across users. I derive a stable single-column key by hashing the pair: `TO_HEX(MD5(CONCAT(user_pseudo_id,`

'-', CAST(ga_session_id AS STRING)))) , then count sessions as COUNT(DISTINCT session_key) rather than COUNT(*) , which would count events. I deliberately use MD5 , not FARM_FINGERPRINT , for a stable cross-environment key, and I cast ga_session_id (an INT64) to STRING before concatenation so the delimiter actually separates the parts. **Why Helios demonstrates it.** int_ga4__sessionized is built on exactly (user_pseudo_id, ga_session_id) with the canonical session_key = TO_HEX(MD5(...)) expression, and sessions = COUNT(DISTINCT session_key) is the one allowed definition repo-wide — it is a keystone that fails silently if wrong. **Follow-ups.** Why MD5 over FARM_FINGERPRINT for a key you persist? / How do you handle events where ga_session_id is NULL? / How would you bound a session by a 30-minute inactivity timeout if you couldn't trust ga_session_id ?

Q3. Write SQL to roll raw GA4 events up to one row per session with session-scoped source/medium.

Ideal answer. I unnest the params I need per event, then aggregate to the session grain, taking the first non-null source/medium ordered by time. ARRAY_AGG(x IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)] cleanly returns the earliest populated value:

```
WITH ev AS (
  SELECT
    user_pseudo_id,
    (SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key='ga_session_id') AS
      ga_session_id,
    event_timestamp,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='source') AS
      p_source,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='medium') AS p_medium
  FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
)
SELECT
  user_pseudo_id, ga_session_id,
  COALESCE(ARRAY_AGG(p_source IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)],
    '(direct)') AS session_source,
  COALESCE(ARRAY_AGG(p_medium IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)],
    '(none)') AS session_medium
FROM ev
WHERE ga_session_id IS NOT NULL
GROUP BY user_pseudo_id, ga_session_id
```

The IGNORE NULLS + LIMIT 1 is the trick: most events carry no source param, so a plain MIN or unfiltered ARRAY_AGG would surface a NULL or an arbitrary value. **Why Helios demonstrates it.** This is the spine of int_ga4__sessionized ; the session-scoped source/medium it produces feeds the channel_group macro, which the Decompose and Diagnose agents lean on to explain funnel movement. **Follow-ups.** Why first-non-null and not last? / How do '(direct)' / '(none)' defaults affect downstream channel grouping? / What if two events share an event_timestamp ?

Q4. The event-level `traffic_source` struct is right there. Why not just group sessions by it?

Ideal answer. In the GA4 export the event-level `traffic_source STRUCT<name, medium, source>` is *user first-touch* attribution — it's stamped from the user's first acquisition and copied onto every later event for that `user_pseudo_id`. A user acquired via Organic Search who returns months later via Email still carries `medium='organic'` on the new events, so grouping sessions by it systematically over-credits acquisition channels and under-credits re-engagement. Worse, it manufactures fake mix-shift that masquerades as rate-change. The correct source is the first non-null `source / medium` from `event_params` within the session itself; event-level `traffic_source` is acceptable only as a documented fallback when session params are entirely NULL, and as the basis for a genuinely user-level `first_touch_channel`. **Why Helios demonstrates it.** Helios codifies this as a hard rule — "never use event-level `traffic_source` for session-scoped channel grouping" — and derives session source/medium from `event_params` in `int_ga4__sessionized`, precisely to stop a phantom Simpson's-paradox confound from polluting the Decompose agent. **Follow-ups.** When is event-level `traffic_source` actually the right thing to read? / How would `collected_traffic_source` change your derivation in newer exports? / How do you test that your session source matches GA4's own UI numbers?

Q5. Write SQL that flags whether a session reached each funnel step, guaranteeing the funnel never inverts.

Ideal answer. I compute per-session "reached" flags as *max-downstream* monotonic flags: a session reached `add_to_cart` if it fired `add_to_cart` or any later event. Doing it monotonically guarantees `sessions ≥ reached_view_item ≥ ... ≥ reached_purchase`, so every step rate is ≤ 1 by construction.

```
SELECT
  session_key,
  MAX(event_name = 'view_item'           OR reached_later) AS reached_view_item,
  MAX(event_name = 'add_to_cart'         OR reached_later) AS reached_add_to_cart,
  MAX(event_name IN ('begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase')) AS
    reached_begin_checkout,
  MAX(event_name = 'purchase')           AS reached_purchase
FROM events_with_session
GROUP BY session_key
```

In practice I implement the monotonicity by including each downstream event in the upstream flag's predicate (as in `reached_begin_checkout` above), which is why the names are `reached_*`, not `did_*`. The funnel is `session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`. **Why Helios demonstrates it.** `int_ga4__funnel_steps` materializes exactly these monotonic `reached_*` flags (the retired `did_*` names are banned for this reason), making the funnel a keystone that downstream rates

can trust. **Follow-ups.** Why are non-monotonic per-step flags dangerous for diagnosis? / How do you handle a session that purchases without an `add_to_cart` event? / Where do you store `session_revenue` so daily rollups keep dollar impact?

Q6. Compute step-to-step conversion rates and overall session conversion from the funnel flags.

Ideal answer. I aggregate the boolean flags with `COUNTIF` and divide adjacent stages, computing each rate as a ratio of summed counts at the group level — never an average of per-session ratios.

```
SELECT
  COUNTIF(reached_view_item) AS view_item_sessions,
  SAFE_DIVIDE(COUNTIF(reached_add_to_cart), COUNTIF(reached_view_item)) AS view_to_cart_rate,
  SAFE_DIVIDE(COUNTIF(reached_begin_checkout), COUNTIF(reached_add_to_cart)) AS
    cart_to_checkout_rate,
  SAFE_DIVIDE(COUNTIF(reached_purchase), COUNTIF(reached_begin_checkout)) AS
    checkout_to_purchase_rate,
  SAFE_DIVIDE(COUNTIF(reached_purchase), COUNT(DISTINCT session_key)) AS
    session_conversion_rate
FROM fct_funnel
```

`SAFE_DIVIDE` prevents divide-by-zero from blanking a whole row, and `COUNTIF(flag)` is cleaner than `SUM(CASE WHEN flag THEN 1 ELSE 0 END)`. `session_conversion_rate = purchasing_sessions / sessions` is the headline. **Why Helios demonstrates it.** These are canonical metrics (`view_to_cart_rate`, `cart_to_checkout_rate`, `checkout_to_purchase_rate`, `session_conversion_rate`) defined once in `semantic_models.yml`; the agent never types this SQL — `semantic-mcp.build_query` emits it from the registry. **Follow-ups.** Why `SAFE_DIVIDE` over `/`? / What does `cart_abandonment_rate` equal in these terms? / How do micro-segment zero-denominators corrupt a naive average-of-ratios?

Q7. Dedup is a classic GA4 trap. How do you guarantee one row per transaction for revenue?

Ideal answer. GA4 can emit duplicate `purchase` events (retries, double-fires), so revenue must be deduped on `transaction_id`. I keep the first event per `transaction_id` with a window function, then sum:

```
WITH ranked AS (
  SELECT
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='transaction_id') AS
      transaction_id,
    ecommerce.purchase_revenue_in_usd AS revenue,
    ROW_NUMBER() OVER (
      PARTITION BY (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE
        ep.key='transaction_id')
```

```

    ORDER BY event_timestamp
  ) AS rn
FROM `... events_*`
WHERE event_name = 'purchase'
)
SELECT transaction_id, revenue FROM ranked WHERE rn = 1 AND transaction_id IS NOT NULL

```

I deliberately exclude NULL/empty `transaction_id` rows (which can't be a real order) and use `_in_usd` revenue so I never aggregate a non-USD twin column. **Why Helios demonstrates it.** `fct_orders` is grain-exactly one row per `transaction_id` with a `unique + not_null` dbt test on the key, and a reconciliation test asserts `SUM(revenue)` matches the source to the cent — the verify-then-trust backbone. **Follow-ups.** `ROW_NUMBER` vs `QUALIFY` vs `DISTINCT` here — when does each break? / How would you detect duplicate transactions before deduping? / Why exclude empty-string `transaction_id`?

Q8. Show how `QUALIFY` simplifies window-function dedup and where it doesn't apply.

Ideal answer. `QUALIFY` filters on a window function without a wrapping subquery, so the Q7 dedup collapses to one statement: `SELECT ... FROM purchases QUALIFY ROW_NUMBER() OVER (PARTITION BY transaction_id ORDER BY event_timestamp) = 1`. It's BigQuery-specific sugar that runs after `SELECT / WINDOW`, the way `HAVING` runs after `GROUP BY`. It does *not* help when I need the deduped set as a CTE consumed multiple times, or when the dedup key itself is a heavy correlated subquery I'd rather compute once — there I keep an explicit CTE for readability and to avoid recomputing the subquery. For "latest record wins" I just flip the `ORDER BY` to `DESC`. **Why Helios demonstrates it.** Helios uses `QUALIFY` for single-pass dedup in marts but keeps explicit CTEs in `int_ga4_sessionized` where the same param extraction feeds several columns — readability and cost both matter under the 5 GiB byte budget. **Follow-ups.** What's the evaluation order of `QUALIFY` relative to `WHERE / GROUP BY / HAVING`? / When does `ROW_NUMBER` vs `RANK` vs `DENSE_RANK` change your result? / How do ties in `ORDER BY` make dedup non-deterministic and how do you fix it?

Q9. Write a cohort-retention query: users grouped by first-active week, retained in later weeks.

Ideal answer. I anchor each user to a cohort by their first activity, then count distinct users active in each subsequent period offset, and divide by the cohort's size.

```

WITH first_seen AS (
  SELECT user_pseudo_id,
         MIN(DATE_TRUNC(DATE(TIMESTAMP_MICROS(event_timestamp)), WEEK)) AS cohort_week
  FROM `... events_*` GROUP BY user_pseudo_id
),
activity AS (
  SELECT DISTINCT user_pseudo_id,

```

```

        DATE_TRUNC(DATE(TIMESTAMP_MICROS(event_timestamp)), WEEK) AS active_week
    FROM `... events_*`
)
SELECT f.cohort_week,
    DATE_DIFF(a.active_week, f.cohort_week, WEEK) AS week_offset,
    COUNT(DISTINCT a.user_pseudo_id) AS retained_users,
    SAFE_DIVIDE(COUNT(DISTINCT a.user_pseudo_id),
        MAX(COUNT(DISTINCT a.user_pseudo_id)) OVER (PARTITION BY f.cohort_week)) AS
        retention_rate
FROM first_seen f JOIN activity a USING (user_pseudo_id)
GROUP BY 1, 2

```

The `MAX( ... ) OVER (PARTITION BY cohort_week)` grabs week-0 size (the largest cohort count) as the denominator, giving a clean triangle. **Why Helios demonstrates it.** Cohort logic is owned by `stats-mcp.cohort_retention` (the only math path), not hand-SQL'd in a prompt; the dataset's ~3-month window is exactly why Helios reports short-horizon retention proxies and explicitly defers true LTV. **Follow-ups.** Why is cohort retention only a proxy on a 90-day window? / How does cookie-grain `user_pseudo_id` (`user_id` mostly NULL) bias retention up or down? / How would you turn this into rolling vs classic retention?

Q10. Explain the mix-vs-rate aggregation rule. Why is `SUM(num)/SUM(den)` not the same as `AVG(rate)`?

Ideal answer. A correct aggregate rate is `SUM(numerator)/SUM(denominator)` computed *after* grouping — equivalently a volume-weighted average of segment rates, $R = \sum_i w_i \cdot r_i$ where w_i is segment volume share. `AVG(per-segment rate)` weights every segment equally regardless of size, so a tiny segment with a wild rate distorts the headline and you get Simpson's paradox: each segment's rate can rise while the blended rate falls because traffic mix shifted toward low-rate segments. So I never average ratios:

```

-- right: SAFE_DIVIDE(SUM(purchasing_sessions), SUM(sessions))
-- wrong: AVG(SAFE_DIVIDE(purchasing_sessions, sessions))

```

This is also why I keep numerator and denominator as separate additive columns through every rollup, deferring the division to the very last step. **Why Helios demonstrates it.** This is Helios's core Simpson's-paradox defense, encoded as a rule ("compute rates as `SUM(num)/SUM(den)` after grouping, never an average of per-segment ratios") and operationalized by `stats-mcp.decompose_change`, which splits ΔR into `mix + rate + interaction`. **Follow-ups.** Show the mix/rate/interaction decomposition algebra. / Why carry numerator and denominator separately to the top of the rollup? / Give a concrete case where every segment improves but the total declines.

Q11. A daily funnel mart must support both trend lines and segment decomposition. How do you structure the aggregation?

Ideal answer. I aggregate to (day × segment-dimensions) and store *additive components* — counts, not pre-divided rates — so any rate can be recomputed by summing num and den across whatever slice the question needs. A row is keyed by day, device_category, channel_group, country etc., carrying sessions, reached_* counts, transactions, and revenue. Rates are computed at read time with SAFE_DIVIDE(SUM(num), SUM(den)). Critically, this mart must aggregate the *session-grain fact* that already carries session_revenue, not the raw funnel-step table, or the dollar figures vanish.

```
SELECT day, device_category, channel_group,
       COUNT(DISTINCT session_key) AS sessions,
       COUNTIF(reached_purchase) AS purchasing_sessions,
       SUM(session_revenue) AS revenue
FROM fct_funnel GROUP BY 1,2,3
```

Why Helios demonstrates it. fct_daily_funnel aggregates fct_funnel (which carries session_revenue) — explicitly not int_ga4__funnel_steps — because dropping revenue here would break the eval's dollar-at-risk labels; this is called out as a keystone. **Follow-ups.** Why must the mart store counts, not rates? / What grain do you choose and how does cardinality affect cost? / How does partitioning by event_date + clustering by device_category, channel_group help here?

Q12. How do you keep BigQuery scans cheap on a date-sharded events_* table?

Ideal answer. The wildcard table is sharded by date, so I prune with _TABLE_SUFFIX BETWEEN '20201101' AND '20210131' to scan only the needed shards — a WHERE on a derived event_date does not prune shards. In the dbt marts (which are partitioned by event_date and clustered by device_category, channel_group) I filter on the partition column to prune partitions and on cluster keys to prune blocks. I never SELECT * on a wide nested table; I project only the params I unnest. And I run a dry_run first to read the byte estimate before executing, narrowing the window or dimensions if it's over budget — never blindly retrying. **Why Helios demonstrates it.** warehouse-mcp is the sole BigQuery client and enforces a mandatory dry_run → run_query byte-budget gate (a fixed 5 GiB/run budget); over-budget queries get narrowed, not retried, and marts are partitioned/clustered exactly as above. **Follow-ups.** Why doesn't WHERE event_date BETWEEN ... prune wildcard shards? / Partitioning vs clustering — what does each prune? / How does dry_run change your query-authoring loop?

Q13. Helios's LLM never hand-writes SQL — semantic-mcp emits it. As an analyst, what does that change about how you work, and what's the tradeoff?

Ideal answer. It moves the craft from "write a query per question" to "encode each metric and dimension *once*, correctly, in a governed registry," then compose validated SQL from those primitives. The payoff is zero hallucinated columns, guaranteed consistent definitions across every brief, and free reconciliation — the same `session_conversion_rate` everywhere. The tradeoff is reduced ad-hoc flexibility: a genuinely novel question needs a registry change (a reviewed PR), not a quick one-off query, and the registry must be expressive enough (metrics × dimensions × filters × windows) to cover real questions or analysts route around it. I accept that friction because correctness and trust beat speed-to-first-query for an autonomous system whose numbers ship to executives. **Why Helios demonstrates it.** `semantic-mcp.build_query` is the *only* path to SQL; it composes from the 28-metric/16-dimension registry in `semantic_models.yml`, so an unknown name is a hard error, not a fallback to free SQL — directly serving the 0-hallucination target. **Follow-ups.** How do you keep the registry expressive without it becoming a SQL-in-YAML escape hatch? / What's your process for adding a new governed metric? / Where does ad-hoc exploration legitimately still happen?

Metric Design

Q14. What is "ratio discipline" and why does every analytics team eventually get burned without it?

Ideal answer. Ratio discipline is the practice of treating every rate as an explicit numerator/denominator pair with a stated grain, and aggregating it as `SUM(num)/SUM(den)` — never storing or averaging a pre-divided ratio. Teams get burned because pre-divided rates can't be re-aggregated (you can't sum percentages across days to get the period rate), and averaging ratios silently triggers Simpson's paradox. The discipline also forces clarity on the denominator: "conversion rate" is meaningless until you say sessions vs users vs add-to-carts. I always answer three questions before a rate ships: what's the numerator, what's the denominator, and at what grain were they counted. **Why Helios demonstrates it.** Every Helios rate metric in `semantic_models.yml` is defined as an explicit numerator/denominator with grain, and the repo rule mandates `SUM(num)/SUM(den)` after grouping — the mix-vs-rate decomposition literally depends on this. **Follow-ups.** Why can't you sum daily conversion rates to get the monthly rate? / Give two metrics that share a numerator but differ only by denominator. / How does this rule interact with weighting in `decompose_change`?

Q15. Walk through choosing the denominator for "conversion rate" in an ecommerce funnel.

Ideal answer. The denominator encodes the question. Session conversion rate uses `sessions` (the acquisition+UX question: of all visits, how many bought). User conversion rate uses distinct users (the demand question, but on GA4 it's cookie-grain since `user_id` is mostly NULL, so it's an approximation). Step rates use the *prior* step's count (`add_to_cart_sessions` /

`view_item_sessions`) to isolate where in the funnel friction lives. Picking the wrong denominator makes a metric directionally misleading: a session-denominated rate falls when bot/direct traffic spikes even if buying behavior is unchanged. I default to session-grain for funnel diagnosis because the question is "what happened to the visits that converted." **Why Helios demonstrates it.** Helios's canonical funnel is session-scoped (`session_conversion_rate = purchasing_sessions / sessions`) with step rates denominated on the prior `reached_*` count, and user-level metrics are explicitly flagged as cookie-based approximations. **Follow-ups.** When is user-denominated conversion the right choice despite cookie-grain noise? / How do you denominate `checkout_to_purchase_rate` and why? / What does a rising session count do to a session-denominated rate?

Q16. What does it mean for a metric to be "governed," and what does a governed definition contain?

Ideal answer. A governed metric is defined exactly once in a versioned registry — name, numerator, denominator/aggregation, the dimensions it's sliceable by, allowed filters, and its grain — and every consumer reads that single definition rather than re-deriving it. Governance means a metric name resolves to one and only one SQL expression system-wide, an unknown name is a hard error (not a silent free-SQL fallback), and changing a definition is a reviewed code change with a clear blast radius. This is what kills "my conversion rate doesn't match yours" arguments: there is one `session_conversion_rate` and it's compiled, not retyped. **Why Helios demonstrates it.** `models/semantic/semantic_models.yml` holds 28 metrics + 16 dimensions, referential-integrity compiled, and is the single source of truth `semantic-mcp` reads; it maps 1:1 to dbt MetricFlow so the governance is mechanically enforced. **Follow-ups.** What's the difference between a metric registry and a BI tool's saved queries? / How does referential-integrity compilation catch a typo'd dimension? / How does this map onto dbt MetricFlow / semantic models?

Q17. What is metric reconciliation and how do you build it into a pipeline?

Ideal answer. Reconciliation asserts that a transformed/aggregated metric equals an independently-computed canonical total within a tight tolerance — proving the marts didn't silently drift from the raw source. I implement it as a test, not a hope: `SUM(revenue)` from the `orders` fact must equal `SUM(purchase_revenue_in_usd)` from the raw export, and the build/finding fails if drift exceeds tolerance. Revenue I reconcile to the cent (zero tolerance); rates and counts to a small percentage. The key is it runs automatically every build and gates the pipeline, so a regression can't ship unnoticed. **Why Helios demonstrates it.** `warehouse-mcp.reconcile` enforces aggregates against canonical totals with a >0.5% drift failing the finding, and `fct_orders` carries a custom singular test reconciling revenue to the source to the cent — the operational form of "verify-then-trust." **Follow-ups.** Why zero tolerance for revenue but a

percentage for rates? / What's a realistic cause of drift that reconciliation catches? / Where in the run does reconcile execute and what does it block?

Q18. A stakeholder says conversion rate dropped 12%. Before diagnosing, what do you check about the metric itself?

Ideal answer. First, relative vs absolute: 12% relative (e.g. 2.0% -> 1.76%) is very different from 12 percentage points. Second, denominator stability — did the rate "drop" only because the denominator (sessions) grew from a traffic surge or a new low-intent channel, i.e. mix-shift, not behavior? Third, definition drift: did a tracking change, a new bot filter, or a redefinition move the metric rather than reality? Fourth, sample size and seasonality: is the window large enough to be significant, and is this just a weekday/holiday pattern? Only after ruling those out do I decompose into mix vs rate and drill the rate effects. This sequencing stops me from "diagnosing" a measurement artifact. **Why Helios demonstrates it.** This is the Helios pipeline order: Monitor detects the anomaly, Decompose splits mix vs rate, then Diagnose drills *rate* effects first (real behavior) before mix (composition artifacts), and the Critic explicitly attacks seasonality and data-quality explanations before a finding ships. **Follow-ups.** How do you tell mix-shift from rate-change numerically? / What guardrail metric would confirm a data-quality artifact? / How big a sample do you need before 12% is significant?

Q19. Define guardrail metrics and give an ecommerce example set.

Ideal answer. Guardrail metrics are health metrics you watch alongside (and counter to) the metric you're trying to move, to ensure a "win" isn't bought by harming something else or by a measurement glitch. For a checkout-conversion experiment, guardrails include AOV and revenue-per-session (did conversion rise only by discounting away margin?), refund/cancellation rate, page-load latency, and add-to-cart rate upstream (did we just shift where the drop-off lives?). They're also data-quality tripwires: a sudden swing in sessions or null-rate flags instrumentation breakage, not behavior. The discipline is to declare guardrails *before* shipping a change so you're not retrofitting an excuse. **Why Helios demonstrates it.** Helios prescribes powered experiment cards with guardrails baked in via `experiment-mcp`, and the Critic's adversarial checks (insufficient sample, data-quality, seasonality) function as automated guardrails before any finding is trusted. **Follow-ups.** Which guardrail catches "conversion up because we discounted"? / How do guardrails differ from the primary metric statistically? / How would you alert on a guardrail breach?

Q20. How do you design a "revenue-at-risk in dollars" metric from a rate change?

Ideal answer. I translate a rate movement into money by multiplying the rate delta by the volume it acts on and the value per conversion: roughly $\text{revenue_at_risk} \approx \Delta(\text{conversion_rate}) \times \text{sessions_in_window} \times \text{AOV}$, but only for the *rate* portion of the change, because mix-driven

movement isn't a behavior problem to fix. I price the counterfactual — "had the rate held, we'd have earned X more" — annualize or window it explicitly, and attach the confidence interval from the significance test so the dollar figure carries uncertainty. Crucially I attribute it to the specific segment the decomposition isolates, not the blended top line, so the number points at an action.

Why Helios demonstrates it. Quantifying revenue-at-risk in dollars is a Helios headline output; it uses `stats-mcp.decompose_change` to isolate the rate effect (not mix) before pricing it, so the dollar figure reflects fixable behavior, and every finding must carry a dollar impact to count as a finding. **Follow-ups.** Why price only the rate effect, not the full ΔR ? / How do you put a confidence interval on the dollar figure? / How would you avoid double-counting risk across overlapping segments?

Q21. How do you avoid vanity metrics, and what makes a metric "actionable"?

Ideal answer. A vanity metric goes up and to the right but doesn't change a decision — total pageviews, raw session counts in isolation. An actionable metric is tied to a lever someone owns, is decomposable to a segment you can target, carries a baseline + significance so you know the move is real, and implies a next step. My test: "if this metric moves, what specifically would we do?" — if there's no answer, it's a reporting number, not a decision metric. I pair every headline rate with the absolute volume and dollar value so a 0.1pp move isn't dressed up as a crisis. **Why Helios demonstrates it.** Helios's fourth non-negotiable is "every finding is actionable" — a finding without a significance test, a dollar impact, and a recommended action is not a finding; the Prescribe agent converts each into a powered experiment card. **Follow-ups.** Give a metric that's vanity in one context and actionable in another. / How does significance testing separate signal from a vanity wiggle? / What's the minimum a metric needs to drive an experiment?

Q22. How do you handle small-denominator / sparse-segment metrics so they don't mislead?

Ideal answer. Tiny denominators produce wildly noisy rates that dominate naive averages and trigger false anomalies, so I (1) never average per-segment ratios — I aggregate `SUM(num)/SUM(den)` ; (2) apply a minimum-sample threshold before a segment's rate is allowed to drive a finding; (3) use a significance test, not just point estimates, so a $1/3 = 33\%$ segment isn't treated like a 30000/90000 one; and (4) suppress or roll up segments below the threshold into an "Other" bucket. The point is to weight by volume and demand evidence proportional to the claim. **Why Helios demonstrates it.** Helios's Critic explicitly refutes "insufficient sample" hypotheses, `stats-mcp.significance_test` gates findings, and the volume-weighted `SUM(num)/SUM(den)` rule stops a sparse segment from hijacking the blended rate. **Follow-ups.** What minimum sample do you require and how do you choose it? / How does a confidence interval on a small segment look vs a large one? / When is rolling up to "Other" the wrong move?

Q23. Two dashboards report different numbers for the same metric. How do you diagnose and prevent it?

Ideal answer. I diff the definitions first: numerator, denominator, grain, filters (bot exclusion, date boundary, timezone), and the dedup rule. The usual culprits are different denominators (sessions vs users), one side averaging ratios while the other sums components, a timezone/date-cutoff mismatch, or one missing the `transaction_id` dedup. The fix isn't to reconcile by hand each time — it's to make both read one governed definition so the metric *can't* diverge, then add a reconciliation test that fails loudly if they do. Prevention beats forensics. **Why Helios demonstrates it.** This is the entire rationale for the semantic registry: one compiled definition per metric in `semantic_models.yml` consumed via `semantic-mcp`, plus `reconcile` as the automated tripwire — so "your number $\neq$ my number" is structurally impossible. **Follow-ups.** Which single mismatch causes the most "different numbers" incidents? / How does timezone/date-boundary choice silently fork a metric? / How would you audit two existing dashboards quickly?

Q24. How do you design a metric to be robust to seasonality and known calendar effects?

Ideal answer. I separate the metric's *definition* from its *interpretation*: the metric stays a clean rate, and I handle seasonality at comparison time — year-over-year or same-day-of-week comparisons, a seasonal baseline/forecast the actual is judged against, and an explicit launch/holiday calendar so a Black Friday spike isn't flagged as an anomaly. I also prefer windows that span whole weeks to avoid weekday-mix artifacts. The metric shouldn't be "deseasonalized" by baking adjustments into its definition (that hides drift); instead the anomaly detector compares against a seasonal expectation. **Why Helios demonstrates it.** Helios keeps a seasonality/launch calendar in `helios_memory`, uses `stats-mcp.forecast` (prophet) to set seasonal baselines, and the Critic specifically attacks "this is just seasonality" before a finding ships — exactly this separation of definition from interpretation. **Follow-ups.** Why not bake a seasonal adjustment into the metric definition? / How does a forecast baseline beat a fixed prior-period comparison? / How do you encode and apply a launch calendar?

Q25. After a fix ships, how do you measure whether it worked when you can't run a clean A/B?

Ideal answer. With observational, historical data I can't randomize, so I use quasi-experimental readbacks: pre/post comparison against the forecasted counterfactual, and difference-in-differences comparing the treated segment to a comparable untreated control segment over the same window (which nets out shared seasonality/macro effects). I pre-register the metric, guardrails, and expected effect size from the original power analysis, then check whether the realized lift clears the significance bar and the confidence interval excludes zero — being explicit that this is causal *evidence*, not proof, and stating the assumptions (parallel trends, no concurrent confound). Honest framing of the design's limits is the senior move. **Why Helios demonstrates it.** Because the GA4 dataset is observational and historical, Helios designs and *sizes* experiments via `experiment-mcp` and runs pre/post and difference-in-differences readbacks rather than live

A/Bs, and the action-tracking loop in `helios_memory` closes the "did-the-fix-work" question.

Follow-ups. What's the parallel-trends assumption in diff-in-diff and how do you check it? / How does pre-registering the effect size guard against p-hacking the readback? / Where does the action-tracking memory store the outcome and how does it inform the next diagnosis?